

Smart Gauge Learning Guide

How the Strava connection, OAuth tokens, Smart Gauge session tokens, and protected API requests work

Beginner-friendly implementation notes for rebuilding this pattern next time

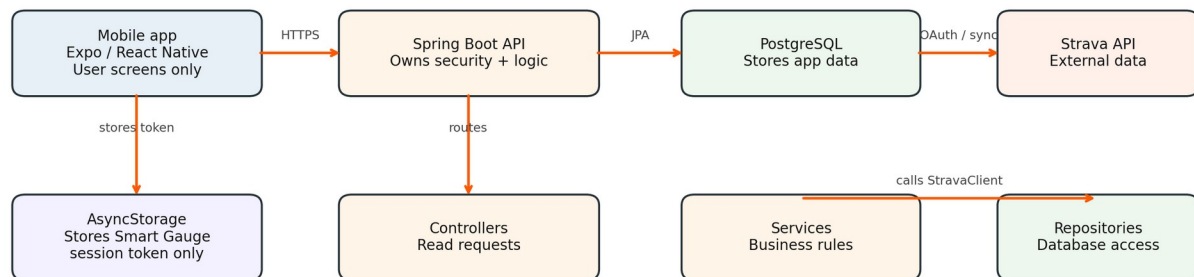
Core idea: Strava tokens let the backend talk to Strava. Smart Gauge session tokens let the mobile app talk to your backend. These are separate responsibilities and should not be mixed.

1. What Smart Gauge Does

Smart Gauge is a mobile endurance app that connects to Strava, imports recent training activities, asks for recovery check-ins, and displays a dashboard. The major learning goal in this stage was not just to make Strava work, but to make it safe enough for beta testers.

- The mobile app is built with Expo / React Native.
- The backend is built with Spring Boot.
- PostgreSQL stores athletes, activities, check-ins, and app sessions.
- Strava is used as the external activity data source.
- TDD is used so controller/service behavior is proven before or while implementing features.

Smart Gauge - Beginner View of the System



2. The Problem We Solved: athleteId Was Not Secure Enough

The early version used athleteId directly in frontend requests. That worked locally because you were the only user. But with beta testers, this creates a security problem.

Old beta-unsafe style:

GET /api/dashboard-information/49461647

Smart Gauge Learning Guide - Strava OAuth and Session Token Reference

POST /api/strava/sync/49461647

If a tester can see or guess another athleteId, they could change the URL and request another person's data. That is why we replaced frontend athleteId usage with backend-generated session tokens.

New safer style:

GET /api/dashboard-information

POST /api/strava/sync

Authorization: Bearer <smart-gauge-session-token>

New rule: the frontend never decides which athlete it is asking for. The backend decides that by looking up the session token.

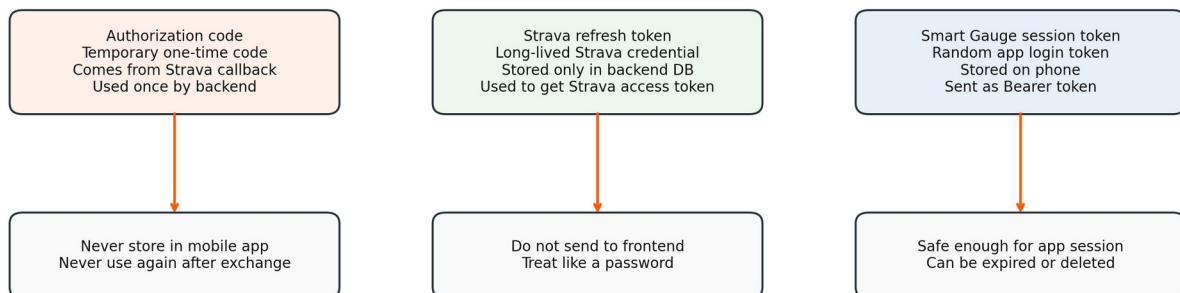
3. OAuth Explained in Plain English

OAuth is a controlled way for Smart Gauge to access Strava data without asking the user for a Strava password. The user signs in on Strava, approves access, and Strava gives your backend permission tokens.

Term	Plain-English meaning	Where it belongs
Strava authorization URL	The link that sends the athlete to Strava to approve access.	Backend builds it; frontend opens it.
Authorization code	A temporary one-time code Strava sends to your backend after approval.	Backend callback only.
Strava access token	Short-lived token used to call Strava API right now.	Backend only.
Strava refresh token	Longer-lived token used to get new Strava access tokens later.	Backend database only.
Smart Gauge session token	Your app's login/session token used by the mobile app to call your backend.	Mobile app stores it; backend validates it.

Do not confuse Strava tokens with Smart Gauge session tokens. Strava tokens are for backend-to-Strava communication. Smart Gauge session tokens are for app-to-backend communication.

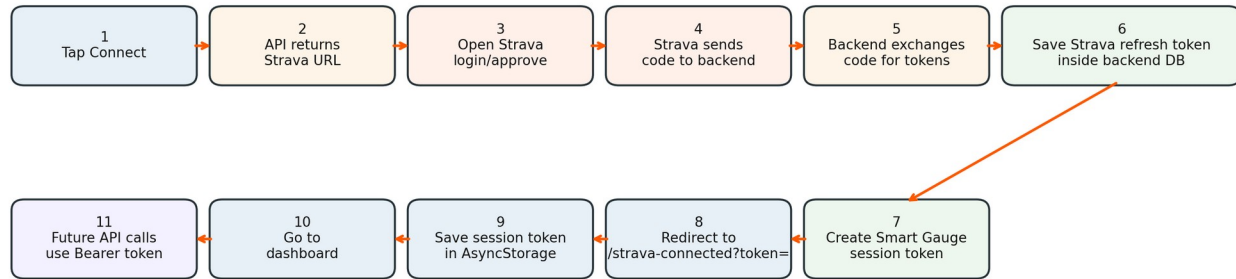
Three Different Token/Code Values - Do Not Mix Them Up



4. Full Strava Connection Flow

This is the complete chain from the user pressing the button to the dashboard being able to load data safely.

Strava Connection Flow - What Happens in Order



4.1 Step-by-Step Breakdown

1. The user taps Connect with Strava in the Expo app.
2. The app calls your backend endpoint GET `/api/strava/authorization-url`.
3. The backend builds the Strava OAuth URL using `clientId`, `redirectUri`, `scope`, and `response_type=code`.
4. The mobile app opens that URL with `Linking.openURL(url)`.
5. The user signs in or approves access on Strava.
6. Strava redirects to your backend callback: `/api/strava/callback?code=abc123`.
7. The backend exchanges the code for Strava tokens.
8. The backend saves or updates the Strava user and refresh token in PostgreSQL.
9. The backend creates a Smart Gauge session token in `smart_gauge_sessions`.
10. The backend redirects to `/strava-connected?token=<sessionToken>`.
11. `strava-connected.tsx` saves the session token in `AsyncStorage`.
12. The app navigates to dashboard.
13. Future dashboard and sync requests send `Authorization: Bearer <sessionToken>`.
14. The backend resolves the session token to `athleteId` and only returns that athlete's data.

5. Frontend: Connect with Strava Button

The Connect button should not build the Strava URL manually. It asks the backend for the URL. This keeps OAuth config in the backend.

```
const handleConnectStrava = async () => {
  try {
    setIsConnecting(true);
    setError("");

    const url = await getStravaAuthorizationUrl();

    if (!url) {
      setError("Could not get Strava authorization URL.");
      return;
    }

    await Linking.openURL(url);
  } catch (error) {
    console.error(error);
    setError("Could not open Strava connection.");
  } finally {
    setIsConnecting(false);
  }
};
```

5.1 getStravaAuthorizationUrl

```
export async function getStravaAuthorizationUrl(): Promise<string> {
  const response = await fetch(`${API_BASE_URL}/api/strava/authorization-url`);

  if (!response.ok) {
    throw new Error("Failed to get Strava authorization URL.");
  }

  return response.text();
}
```

Learning point: this endpoint returns plain text, not JSON. That is why the frontend uses `response.text()` instead of `response.json()`.

6. Why strava-connected.tsx Exists

`strava-connected.tsx` is a small processing route. It receives the Smart Gauge session token after the backend redirects from the Strava callback. It is not meant to be a normal screen the user stays on.

Backend redirect target during local Expo web testing:
`http://localhost:8081/strava-connected?token=<sessionToken>`

Backend redirect target for native/TestFlight later:
`smartgauge://strava-connected?token=<sessionToken>`

When Expo Router sees `/strava-connected`, it loads `src/app/strava-connected.tsx`. That file reads token from the URL, saves it, then routes to dashboard.

```
export default function StravaConnectedScreen() {
```

```

const params = useLocalSearchParams();

useEffect(() => {
  async function saveConnection() {
    const tokenParam = Array.isArray(params.token)
      ? params.token[0]
      : params.token;

    if (!tokenParam) return;

    await saveSessionToken(tokenParam);
    router.replace("/dashboard");
  }

  saveConnection();
}, [params.token]);

return <Text>Connecting Strava...</Text>;
}

```

Learning point: without strava-connected.tsx, the app would receive a URL with a token but have no route responsible for saving it.

7. Backend: Authorization URL Endpoint

The backend returns the URL that sends the user to Strava. This URL includes `client_id`, `response_type`, `redirect_uri`, `approval_prompt`, and `scope`.

```

@GetMapping("/authorization-url")
public String getAuthorizationUrl() {
    return stravaService.getAuthorizationUrl();
}

public String getAuthorizationUrl() {
    return UriComponentsBuilder
        .fromUriString("https://www.strava.com/oauth/authorize")
        .queryParams("client_id", stravaProperties.clientId())
        .queryParams("response_type", "code")
        .queryParams("redirect_uri", stravaProperties.redirectUri())
        .queryParams("approval_prompt", "force")
        .queryParams("scope", "read,activity:read_all")
        .build()
        .toUriString();
}

```

Parameter	Why it exists
<code>client_id</code>	Tells Strava which application is requesting access.
<code>response_type=code</code>	Tells Strava to return a temporary authorization code.
<code>redirect_uri</code>	Tells Strava where to send the user after approval.
<code>approval_prompt=force</code>	Forces Strava to show the approval screen again, useful while testing.
<code>scope=read,activity:read_all</code>	Requests permission to read athlete activity data.

8. Backend: Callback and Token Exchange

After the athlete approves access, Strava calls your backend callback with a code. The backend exchanges the code for Strava tokens.

```
@GetMapping("/callback")
public ResponseEntity<Void> handleCallback(@RequestParam String code) {
    Long athleteId = stravaService.exchangeAuthorizationCode(code);
    String token = sessionService.createSessionForAthlete(athleteId);

    URI redirectUri = URI.create(
        smartGaugeProperties.appRedirectUri() + "?token=" + token
    );

    return ResponseEntity
        .status(HttpStatus.FOUND)
        .location(redirectUri)
        .build();
}
```

8.1 What exchangeAuthorizationCode Does

1. Sends the authorization code to Strava token endpoint.
2. Receives access token, refresh token, expiration, and athlete profile.
3. Gets athleteId from response. Example: response.athlete().id().
4. Finds existing StravaUserEntity by athleteId or creates a new one.
5. Updates firstname, lastname, refreshToken, and updatedAt.
6. Saves the StravaUserEntity.
7. Returns athleteId to the controller.

Important: athleteId should be the primary key for StravaUserEntity and should not use @GeneratedValue. Strava supplies this ID.

```
StravaUserEntity user = stravaUserRepository.findById(athleteId)
    .orElseGet(() -> StravaUserEntity.builder()
        .athleteId(athleteId)
        .connectedAt(LocalDateTime.now())
        .build());

user.setFirstname(response.athlete().firstname());
user.setLastname(response.athlete().lastname());
user.setRefreshToken(response.refreshToken());
user.setUpdatedAt(LocalDateTime.now());

stravaUserRepository.save(user);
return athleteId;
```

9. Backend: Creating a Smart Gauge Session Token

After Strava connection succeeds, the backend creates its own session token. This is not a Strava token. It is a random Smart Gauge token that identifies the current app session.

```
public String createSessionForAthlete(Long athleteId) {
    StravaUserEntity stravaUser = stravaUserRepository.findById(athleteId)
        .orElseThrow(() -> new IllegalArgumentException("Strava user not found."));

    String token = UUID.randomUUID().toString();

    SmartGaugeSessionEntity session = SmartGaugeSessionEntity.builder()
        .token(token)
        .stravaUser(stravaUser)
        .createdAt(LocalDateTime.now())
        .expiresAt(LocalDateTime.now().plusDays(30))
        .build();

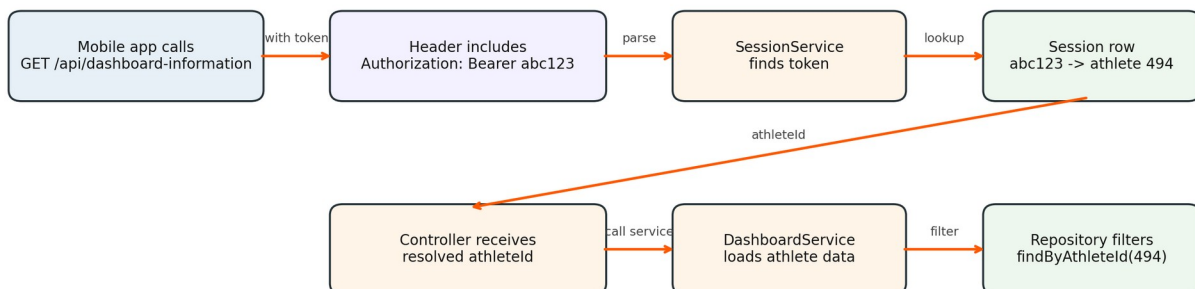
    sessionRepository.save(session);
    return token;
}
```

9.1 Why Create Our Own Session Token?

- The mobile app should not store the Strava refresh token.
- The frontend should not decide which athleteId it is allowed to access.
- The backend can expire or delete app sessions.
- The same pattern can later support real login, logout, revoke, and multi-device sessions.
- It makes the API safer for beta testers because athleteId disappears from frontend URLs.

10. How Protected Requests Work

How a Protected Request Works



10.1 SessionService Resolves the Token

```
public Long getAthleteIdFromAuthorizationHeader(String authorizationHeader) {
    String token = authorizationHeader.replace("Bearer ", "");
    return getAthleteIdFromToken(token);
}

public Long getAthleteIdFromToken(String token) {
    SmartGaugeSessionEntity session = sessionRepository.findById(token)
        .orElseThrow(() -> new IllegalArgumentException("Invalid session token."));

    if (session.getExpiresAt().isBefore(LocalDateTime.now())) {
        throw new IllegalArgumentException("Session expired.");
    }

    return session.getStravaUser().getAthleteId();
}
```

10.2 DashboardController Uses the Resolved athleteId

```
@GetMapping
public List<DashboardFeedRecord> getDashboardFeedRecord(
    @RequestHeader("Authorization") String authorizationHeader
) {
    Long athleteId = sessionService
        .getAthleteIdFromAuthorizationHeader(authorizationHeader);

    return dashboardService.getAllActivitiesForDashboard(athleteId);
}
```

10.3 StravaController Sync Uses the Resolved athleteId

```
@PostMapping("/sync")
public List<ActivityResponseRecord> syncLastSevenActivities(
    @RequestHeader("Authorization") String authorizationHeader
) {
    Long athleteId = sessionService
        .getAthleteIdFromAuthorizationHeader(authorizationHeader);

    return stravaService.syncLastSevenActivities(athleteId);
}
```

Learning point: the controller receives the header, but StravaService still receives athleteId. The controller/security layer converts token -> athleteId first.

11. Frontend: Calling Protected Endpoints

After the token is saved, the frontend gets it from AsyncStorage and sends it in the Authorization header.

```
export async function getDashboardFeed(): Promise<DashboardFeedItem[]> {
    const token = await getSessionToken();

    if (!token) {
        throw new Error("Missing session token.");
    }
}
```



```

const response = await fetch(`${API_BASE_URL}/api/dashboard-information`, {
  headers: {
    Authorization: `Bearer ${token}`,
  },
});

if (!response.ok) {
  throw new Error("Failed to fetch dashboard feed.");
}

return response.json();
}

export async function syncStravaActivities(): Promise<void> {
  const token = await getSessionToken();

  if (!token) {
    throw new Error("Missing session token.");
  }

  const response = await fetch(`${API_BASE_URL}/api/strava/sync`, {
    method: "POST",
    headers: {
      Authorization: `Bearer ${token}`,
    },
  });

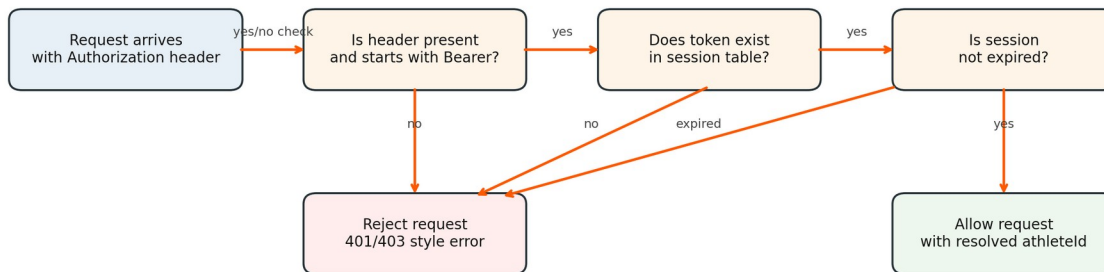
  if (!response.ok) {
    throw new Error("Failed to sync Strava activities.");
  }
}

```

12. Session Validation Decision Tree

A protected endpoint should only continue if the request has a valid, unexpired session token. Otherwise, the backend should reject the request.

Session Token Validation Decision Tree



13. Data Model for Tokens and Activities

Table/entity	Key fields	Purpose
strava_users	athleteId, firstname, lastname, refreshToken, connectedAt, updatedAt	One row per Strava athlete connected to Smart Gauge.
smart_gauge_sessions	token, stravaUser, createdAt, expiresAt	Maps a random app session token to a Strava athlete.
activities	id, athleteId, externalSource, externalActivityId, sportType, startDate, distanceMeters	Stores imported Strava activities. athleteId is required for multi-user safety.
recovery_checkins	id, activity, rpe, painScore, painLocation, mood, note, createdAt	Stores subjective recovery data linked to an activity.
Rule: every user-owned table should either directly include athleteId or be linked to a row that belongs to an athlete. That is how the backend enforces data separation.		

14. How to Test This Pattern Next Time

The safest way to implement this again is to write tests at each layer. Start with the controller behavior, then service behavior.

14.1 StravaController Sync Test

This test proves the controller reads the Authorization header, asks SessionService for athleteId, and then calls StravaService with athleteId.

```
@Test
void shouldSyncStravaActivitiesUsingSessionToken() throws Exception {
    ActivityResponseRecord response = new ActivityResponseRecord(
        1L, "STRAVA", "12345", "RUN", "Morning Run",
        LocalDateTime.of(2026, 7, 9, 8, 0),
        5.0, 40, "8:00"
    );
}
```

```

when(sessionService.getAthleteIdFromAuthorizationHeader("Bearer test-token"))
    .thenReturn(12345L);

when(stravaService.syncLastSevenActivities(12345L))
    .thenReturn(List.of(response));

mockMvc.perform(post("/api/strava/sync")
    .header("Authorization", "Bearer test-token"))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.id").value(1))
    .andExpect(jsonPath("$.externalSource").value("STRAVA"));

verify(sessionService).getAthleteIdFromAuthorizationHeader("Bearer test-token");
verify(stravaService).syncLastSevenActivities(12345L);
}

```

Why we do not mock syncLastSevenActivities("Bearer test-token"): StravaService should not know about HTTP headers. It should receive athleteId after SessionService resolves the token.

14.2 SessionService Tests

Test	Purpose
shouldCreateSessionForAthlete	Proves a Strava user can be turned into a Smart Gauge session token.
shouldReturnAthleteIdFromValidToken	Proves a valid token resolves back to the correct athleteId.
shouldThrowExceptionForInvalidToken	Proves fake tokens cannot access data.
shouldThrowExceptionForExpiredToken	Proves expired tokens cannot access data.

15. Implementation Checklist for Next Time

1. Create the external OAuth app and obtain clientId/clientSecret.
2. Create backend properties for external clientId, secret, redirectUri, and appRedirectUri.
3. Add GET /authorization-url endpoint that builds the external authorization URL.
4. Add GET /callback endpoint to receive code from the external provider.
5. Exchange the code for external tokens inside backend service/client code.
6. Store the external refresh token in the backend database only.
7. Create your own app session token and store token -> user mapping.
8. Redirect to frontend callback route with the app session token.
9. Frontend callback route saves app session token in storage.
10. All protected frontend API calls send Authorization: Bearer <sessionToken>.
11. Backend protected controllers resolve session token to user/athlete ID.
12. Services and repositories filter data by that resolved user/athlete ID.
13. Write controller tests proving Authorization header is used.
14. Write service tests proving token validation and expiration behavior.

15. Before beta testing, remove public user IDs from frontend URLs.

16. Render / Beta Hosting Notes

When using Render for beta testers, local addresses must be replaced with hosted URLs. Testers cannot use your localhost.

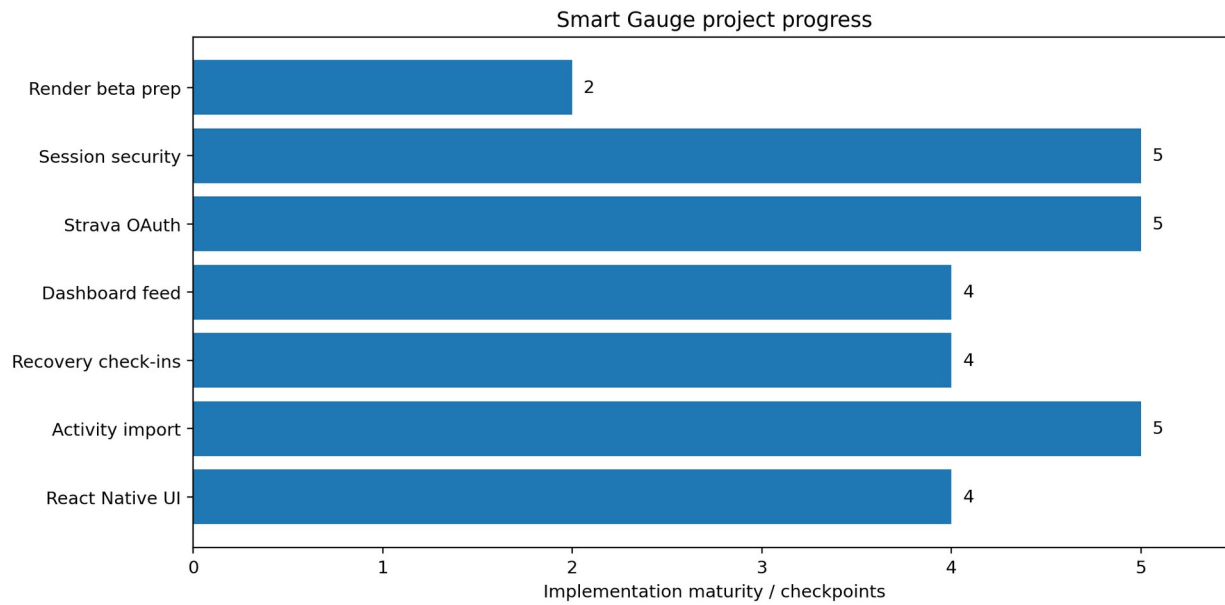
Local development	Render/TestFlight beta
API_BASE_URL=http://localhost:8080	API_BASE_URL=https://your-render-service.onrender.com
STRAVA_REDIRECT_URI=http://localhost:8080/api/strava/callback	STRAVA_REDIRECT_URI=https://your-render-service.onrender.com/api/strava/callback
SMART_GAUGE_APP_REDIRECT_URI=http://localhost:8081/strava-connected	SMART_GAUGE_APP_REDIRECT_URI=smartgauge://strava-connected
Strava callback domain=localhost	Strava callback domain=your-render-service.onrender.com

17. Common Errors and What They Mean

Error/symptom	What it usually means	Fix
redirect_uri invalid	Strava callback domain or redirectUri does not match.	Set Strava callback domain to host only and STRAVA_REDIRECT_URI to full backend callback URL.
URL contains \${STRAVA_REDIRECT_URI}	Spring did not load env vars.	Load environment variables before bootRun or configure Render env vars.
/strava-connected route not found	Expo Router has no matching file.	Create src/app/strava-connected.tsx with a default export.
No bean of type SessionService	Controller dependency was added but not mocked in @WebMvcTest.	Add @MockitoBean private SessionService sessionService;
athlete_id is null	athleteId was not passed from StravaService to ActivityService/entity builder.	Add athleteId to request record and .athleteId(request.athleteId()) in entity builder.
Dashboard returns another user's data	Endpoint still accepts athleteId from frontend or repositories do not filter by athleteId.	Remove athleteId URL path and resolve athleteId from session token.

18. Project Recap

Smart Gauge now has the foundation for a safer beta: Strava OAuth connection, backend token storage, app session tokens, protected dashboard and sync requests, activity import, recovery check-ins, and dashboard presentation.



Final learning summary: OAuth lets Smart Gauge get permission from Strava. Strava refresh tokens stay in the backend. Smart Gauge session tokens are given to the mobile app. Every protected request uses the session token, and the backend resolves it to the correct athleteId before loading data.